Jude Shin
CSC 4888-S01-2268
Homework 3
October 7, 2026

Book Search

1 Explanation

1.1 Fitting the `KNN` Model

The code pre-processes the images and queries into gray-scale, essentially creating a useable vector (or embedding). Then features were extracted from the image. The features are in a high dimensional vector, and those are clustered using `KMeans`. We will be searching on the `KMeans` centroids, not every document. This makes searching our database more efficient.

Each of these clusters (or centroids) are converted into documents (aka, "words"). Finally, these "words" are converted into a histogram. This is the final form that we will be comparing. We save all of these histograms to the K-Nearest-Neighbors model using the `.fit()` function. Our "search database" is finally ready to be used with real world images.

1.2 Using the `KNN` Model

We use the same pre-process pipeline to get a histogram (or "word"). Once we get an embedding that we can compare against, we get the closest 5 "words" that are matches in our fitted `KNN`. The key is that we have to use the same `KMeans` clustering model to get the same words that our `KNN` is fitted on. For each of the results that we got from the `KNN` query, we just plot them out in a for loop in the end.

2 Issues

I got a bunch of weird results because I put everything to create a "word" in a separate function (including the `KMeans` model). In turn, I used a completely new `KMeans` object, resulting in completely different (and irrelevant) words. I just got rid of the function entirely and just had some duplicated code.

I also ended up running out of compute time on Google Colab, so I wrote everything in one python script and ran it on my laptop. It took forever, and when I messed something up, I would have to redo everything. This was tedious and time consuming, but thankfully I didn't have to do that too many times. This is why I added `tqdm` to track the progress of each step.

3 Discussion Questions

1. *Evaluate how accurately the system is able to find the books in the query images.*

   It was pretty surprised with the accuracy of our model. For the first query, there were a lot of dogs in the photo that were similar, but not exactly a match with the query book. I also noticed that the feature detector that we created was pretty good at recognizing fonts. Titles are the most prominent on books; titles also have to get good contrast because they need to be readable, so I guess that is why there was good features that were extracted from them. Figures 1, 2, 3, 4, 5, 6 show all 6 queries respectively.

2. *What do you notice about the search results that are not the same book as the query image? Are these search results related to the search query in some way?*

   Not in the same way (like text on the title of the book or such), but as I said before, the general theme could be seen in the images. Patterns like stars or animals or shapes are preserved through the covers. I thought that the results from the book "Are You My Mother" was pretty funny. The results had a lot of books with dogs on the picture, but only one out of the five were a correct match. I thought that was interesting.

3. *The two images in extra-queries are of books not included in the database. Test what happens when you search for these books.*

   The program doesn't just crash or something; it is pretty good at generalizing features. The themes that were talked about in the previous question are translated to these "unseen" images. The "Piano" book seen in Figure 7 got a similar piano teaching book on rank 5, which was cool. It wasn't the exact same book, but it had a similar graphic of a piano with wings.

   The other query seen in Figure 8 image had a lot of sharp bold text that took up a good chunk of the cover. The 5 matches that were returned had all of the same features mentioned before, however, none were the same book.

4. *Can you think of a way to automatically determine if the search was successful, or if the book being searched is not in the database?*

   We could look at the distance and add some threshold. If the `KNN` algorithm gives the closest match as being extremely large, then we can assume that query was not a known book.

4 Sources Used

I used the `skimage` documentation for the `KMeans` and the `TfidfVectorizer` packages. I had some issues with when to use `TfidfVectorizer.fit_transform` and `TfidfVectorizer.transform`, so I had to do some digging on Google.



Figure 1: The third ranked result was a correct match.

I also had some issues with running this code in Google Colab; I ran out of compute time with the number of images that we were processing.



Figure 2: The first two ranked results were correct matches.



Figure 3: The first ranked result was a correct match.

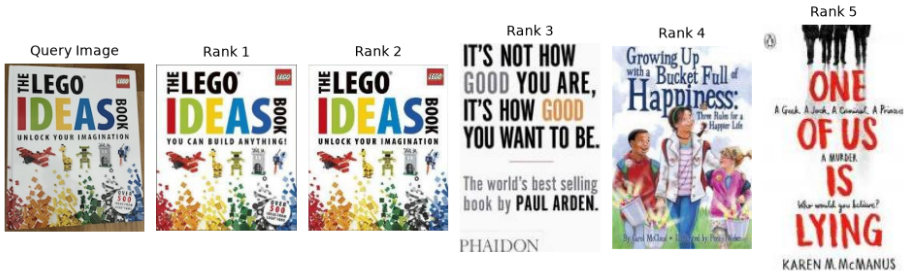


Figure 4: The first two ranked results were correct matches.



Figure 5: The first two ranked results were correct matches.



Figure 6: The first ranked result was a correct match.



Figure 7: The same piano with wings can be seen in result with rank 5.



Figure 8: A common pattern of large bold text can be seen across most of the results.